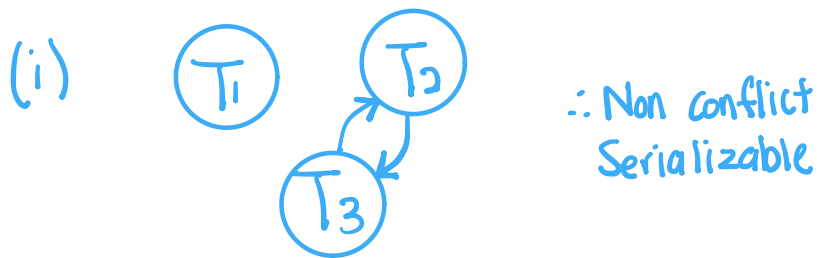


Question 1



(ii) The presence of cycle $T_2 \rightarrow T_3, T_3 \rightarrow T_2$ shows that the concurrent schedule is not conflict serializable. The cycle implies a logical contradiction in execution order, which can lead to dirty read or lost update

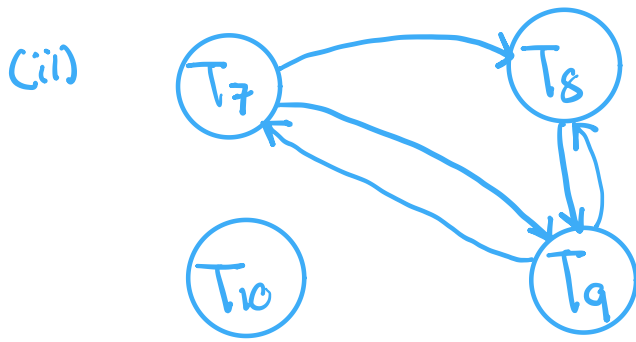
(iii)

T_1	T_2	T_3
Begin $S_lock(x)$ $read_item(n)$ $unlock(n)$ $S_lock(m)$ $read_item(n)$ $unlock(y)$	Begin $x_lock(m)$ $read_item(m)$ $S_lock(N)$ wait	Begin $x_lock(N)$ $read_item(m)$ $S_lock(m)$ wait

(iv) $\therefore$ There is a deadlock since a circular waiting exists in T_2 and T_3 . Besides, the starvation will arise because the transaction is perpetually delayed.

Question 3a

(i) Non Serial. It allows multiple transaction to interleave before it completely end.



(iii) Non conflict serializable. The precedence graph isn't a cycle. It contains cycle at $T_7 \rightarrow T_9$, $T_9 \rightarrow T_7$, $T_8 \rightarrow T_9$ and $T_9 \rightarrow T_8$.

(iv) Lost updates. Multiple transaction concurrently read and write data item u.

Question 3b

(i)

T_1	T_2	T_3	T_4
begin n-lock(A) read(A) $A = A + 200$ write(A) unlock(A) commit	begin n-lock(B) read(B) $B = B - 50$ write(B) unlock(B) n-lock(A) wait read(A) $A = A * 2$ write(A) commit	begin n-lock(A) wait wait read(A) $A = A - 30$ unlock(A) Commit	begin n-lock(C) read(C) write(C) unlock(C) Commit

(ii) Starvation, Livelock, Deadlock